



Facultad de Ciencias Exactas, Ingeniería y Agrimensura (FCEIA)

Tecnicatura Universitaria en Inteligencia Artificial (TUIA)

Procesamiento de Imágenes I - IA 4.4

Trabajo Práctico N° 1 - Año 2026 - 2° Cuatrimestre

Alumnos: Arias Federico (A-4715/5), Bousoño Guillermina (B-6658/3), Gimenez Valentin (G-6103/4), Aguilera Joaquín (A-4417/2)

Fecha: Octubre 2026

Problema 1: Ecualización local de histograma

El problema

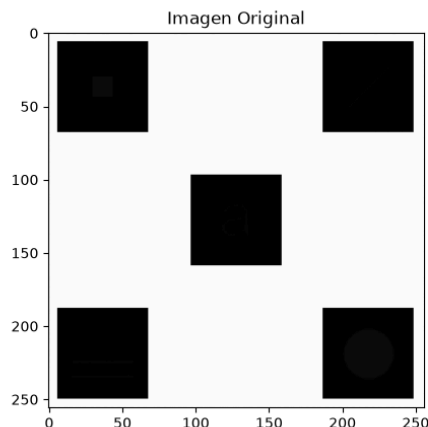
El objetivo del Problema 1 requiere implementar un algoritmo de ecualización local de histograma y usarlo para descubrir qué detalles hay escondidos en la imagen

[Imagen_con_detalles_escondidos.tif](#).

A simple vista la imagen parece estar compuesta por un fondo gris claro (256x256 píxeles, con valores cercanos a 227) y varios cuadrados negros. Sin embargo, dentro de esos cuadrados hay objetos cuyo nivel de intensidad es muy parecido al del fondo local, por lo que el ojo no llega a distinguirlos.

La idea de la técnica es definir una ventana de procesamiento de tamaño $M \times N$ y desplazar su centro píxel por píxel sobre toda la imagen. En cada posición se calcula el histograma de los píxeles que caen dentro de la ventana y, a partir de él, la transformación de ecualización que define el nuevo valor del píxel central. El enunciado nos pide:

- Desarrollar una función que reciba la imagen y el tamaño de la ventana $M \times N$.
- Aplicarla sobre la imagen e informar cuáles son los detalles ocultos.
- Analizar cómo influye el tamaño de la ventana en el resultado.

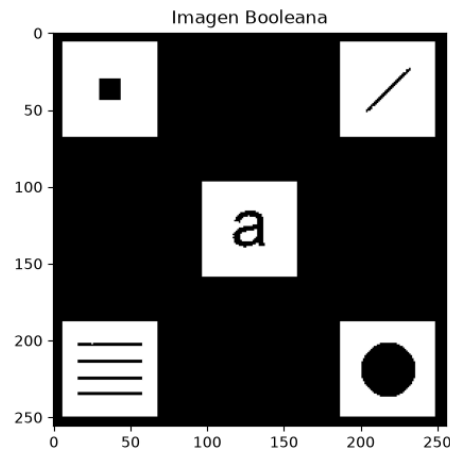


La matriz booleana

Antes de ecualizar nada, quisimos confirmar que realmente había algo escondido. Para eso generamos una matriz booleana que marca solo los píxeles con intensidad menor a 2, es decir, los que son prácticamente negros:

```
img_zeros = img < 2
```

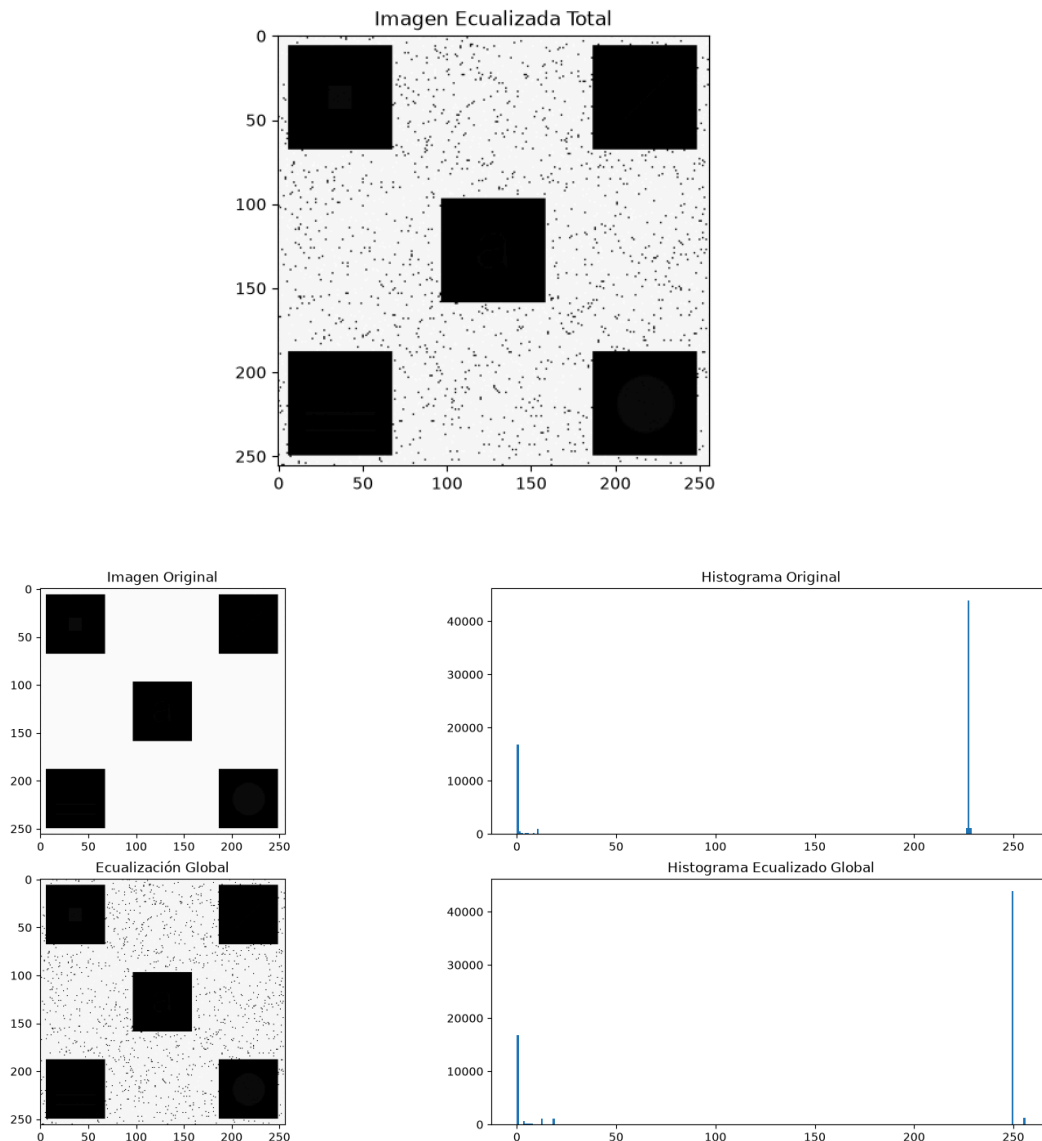
Al graficarla, los objetos ocultos aparecen como “huecos” dentro de los cuadrados negros. Esto nos confirmó que los detalles existen y que la diferencia de intensidad con su entorno es de apenas unos pocos niveles de gris (entre 0 y 11 aproximadamente).



Para entender la dimensión del problema decidimos primero ecualizar la imagen completa con la función de OpenCV y mirar tanto el resultado como su histograma:

```
ecua = cv2.equalizeHist(img)
plt.figure(), plt.imshow(ecua, cmap='gray'), plt.title("Imagen Ecualizada Total"),
plt.show(block=False)
plt.figure(), plt.hist(ecua.flatten(), 256, [0, 256]), plt.title("Ecualizacion"),
plt.show(block=False)
```

Acá nos dimos cuenta de que la redistribución de los niveles de intensidad no favorece a las zonas donde están los objetos ocultos. Como el histograma se calcula con todos los píxeles de la imagen, el fondo claro domina la transformación y la información local se pierde. En lugar de resaltar los detalles, la ecualización reparte los píxeles de forma que termina “ensuciando” la imagen, con ruido en los cuadrados negros pero sin que los objetos se lleguen a distinguir con claridad.



La ventana de procesamiento y el problema de los bordes

Al empezar a trabajar en la función `ecualizacion_local_histograma` nos encontramos con el primer obstáculo: cómo ubicar la ventana de procesamiento centrada en el píxel que queremos analizar. Si colocamos el centro de la ventana en el primer píxel de la matriz, parte de la ventana queda afuera de la imagen.

Para resolverlo, y siguiendo la ayuda que da el propio enunciado, usamos la función de OpenCV `copyMakeBorder`, que agrega un contorno de píxeles nuevos alrededor de la imagen original.

El siguiente desafío fue decidir cuántas filas y columnas agregar. Optamos por la división entera de las filas (M) y de las columnas (N) de la ventana entre 2. Ese valor es justamente lo

que “sobresale” la ventana cuando ponemos su centro en el primer píxel de la imagen. Por ejemplo, con una ventana de 15x15 se agregan 7 píxeles de cada lado.

```
M,N = MxN
# Creamos márgenes en los costados de la imagen que es la mitad de la ventana de
procesamiento MxN
top = M // 2
bottom = top
left = N // 2
right = left

imagen_borde = cv2.copyMakeBorder(imagen,top,bottom,left,right,borderType=border)
```

Como `borderType` por defecto elegimos `cv2.BORDER_REPLICATE`, que extiende hacia afuera el valor de intensidad de los píxeles del contorno original. Nos pareció la opción más fiel para ecualizar, evitando introducir valores ajenos (como pasaría con un borde constante en negro).

Recorrer la imagen

Lo más complicado del problema fue encontrar la manera de recorrer la imagen. Optamos por dos bucles `for` que van sobre el rango de filas y columnas de la imagen original:

```
# Copia de la imagen original
imagen_ecualizada = imagen.copy()

# Empezamos a recorrer la imagen
for i in range(imagen.shape[0]):
    for j in range(imagen.shape[1]):
        # Ventana MxN centrada en el píxel (i, j) original
        ventana = imagen_borde[i : i + M, j : j + N]
        ventana_eq = cv2.equalizeHist(ventana)
        # Solo se guarda el píxel central de la ventana ecualizada
        imagen_ecualizada[i,j] = ventana_eq[M // 2, N // 2]

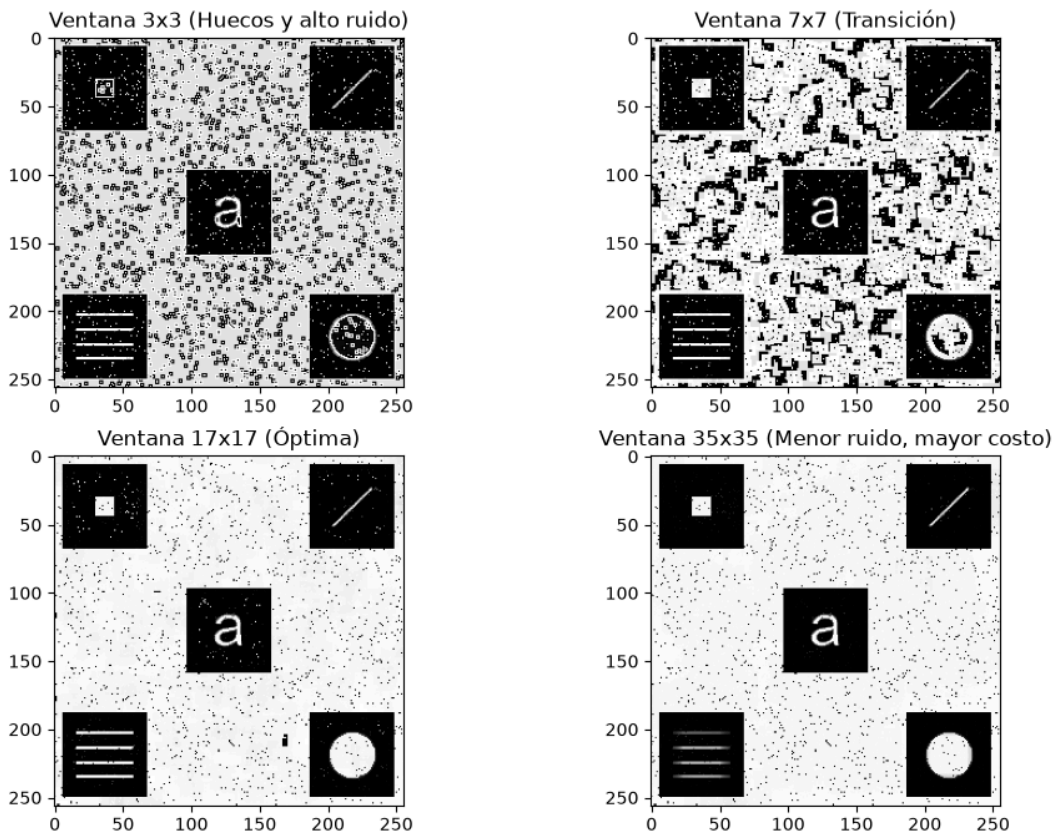
return imagen_ecualizada
```

La clave está en la indexación. Como la imagen con borde tiene $M//2$ filas y $N//2$ columnas extra de cada lado, el recorte `imagen_borde[i : i + M, j : j + N]` sobre la imagen ampliada corresponde exactamente a una ventana $M \times N$ centrada en el píxel (i, j) de la imagen original. Así no hace falta sumar ni restar desplazamientos a mano: el propio borde se encarga de centrar la ventana.

En cada posición se ecualiza la ventana con `cv2.equalizeHist`, pero de la ventana ecualizada solo nos quedamos con el píxel central, `ventana_eq[M // 2, N // 2]`, que se guarda en la posición (i, j) de `imagen_ecualizada`. Como esa salida arranca como una copia de la imagen original, el resultado conserva el mismo tamaño que la original y no el de la imagen con borde. Con estas técnicas de indexación logramos llegar al resultado esperado.

Resultado

Punto c: Influencia del tamaño de la ventana (MxN)



Realizamos 4 imágenes con ecualización local con distintos tamaños de ventana.

- **Ventanas pequeñas (3×3):** Al contar con una muestra de píxeles tan reducida, la estadística local se vuelve inestable y amplifica variaciones mínimas imperceptibles del fondo claro original, generando un intenso efecto de ruido granulado (sal y pimienta) en toda la imagen. Además, cuando el área de 3×3 cae completamente dentro de figuras sólidas anchas (como el círculo o el cuadrado interno), la falta de contraste interno hace que el algoritmo devuelva valores oscuros, dejando a los objetos con apariencia de estar "huecos".
- **Ventanas de transición (7×7):** Se observa una disminución en la densidad del ruido del fondo claro, pero continúan presentándose artefactos oscuros indeseados. Asimismo, las figuras sólidas grandes aún no logran rellenarse de blanco de manera uniforme.
- **Ventana óptima (17×17):** Representa el punto de equilibrio adecuado para las características espaciales de esta imagen en particular. El tamaño es lo suficientemente grande como para abarcar al mismo tiempo la figura interna y su fondo oscuro inmediato, logrando que los elementos sólidos se rellenen de blanco por completo. Paralelamente, estabiliza la estadística del fondo claro, eliminando casi la totalidad del ruido visual presente en las ventanas más pequeñas.
- **Ventanas grandes (35×35):** Aunque logran suavizar un poco más el fondo claro, elevan de manera cuadrática el costo computacional del procesamiento. Si la ventana se

amplía demasiado, comienza a comportarse como un filtro global, mezclando áreas espacialmente distantes y corriendo el riesgo de generar artefactos de borde severos al interpolar objetos de características muy disímiles.

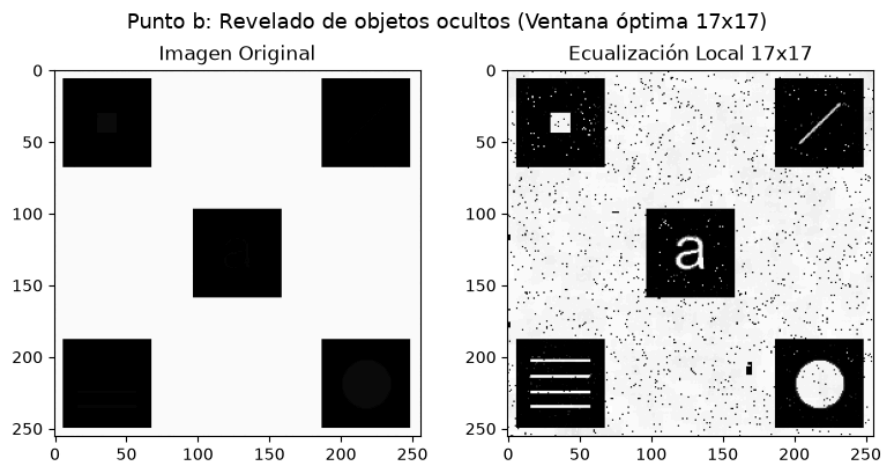
Se identificaron:

- **Zona superior izquierda:** Un cuadrado sólido pequeño.
- **Zona superior derecha:** Una línea diagonal recta ascendente.
- **Zona central:** La letra "a" en formato minúscula.
- **Zona inferior izquierda:** Cuatro líneas horizontales paralelas.
- **Zona inferior derecha:** Un círculo sólido.

Conclusiones

Aplicando la función con una ventana de 17x17, los objetos que antes estaban escondidos dentro de los cuadrados negros aparecen con claridad. A diferencia de la ecualización global, acá cada píxel se transforma según el histograma de su vecindario, por lo que esas pequeñas diferencias de intensidad se estiran y se vuelven visibles.

```
kernel_optimo = (17, 17)
img_optima = ecualizacion_local_histograma(img, kernel_optimo)
```



Problema 2: Validación de planilla de calificaciones

El problema

El objetivo del problema 2 es verificar de forma automática planillas de calificaciones escaneadas como imagen. Cada planilla tiene una tabla de 20 registros y cada registro tiene seis campos a validar: Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final.

Se requiere verificar que cada campo cumpla con las restricciones de formato:

- **Legajo:** exactamente 8 caracteres formando una única palabra.
- **Nombre y apellido:** al menos dos palabras y no más de 12 caracteres.
- **Parciales 1, 2 y 3:** 1 o 2 caracteres consecutivos sin espacios.
- **Condición Final:** un único carácter.

A partir de esto el algoritmo debe:

- Mostrar por terminal qué campos están OK y cuales MAL en cada registro.
- Generar un única imagen con el nombre de los alumnos no aprobados (Condición “L” o “R”) cuyos registros estén correctos, con un indicador que diferencie a quienes recuperan de quienes quedan libres.
- Guardar un CSV con el resultado de cada validación.
- Aplicar el procedimiento para todas las planillas.

Imagen original

Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						

Detección de líneas de la tabla

El primer paso fue binarizar la imagen para separar letras y líneas del fondo. Al analizar los niveles de gris de la planilla encontramos:

- Trazos con valores cercanos a 0 (grises intermedios en los bordes).
- Interior blanco de las celdas cercano a 255.
- Fondo gris de la hoja fuera de la tabla aproximadamente 200.



Suma por filas y columnas

Sumamos los píxeles de tinta de cada fila y de cada columna. Las líneas de la tabla atraviesan casi todo el alto de la tabla. Definimos como línea toda fila o columna cuya suma supere la mitad del valor máximo. El resultado es una señal de pulsos donde cada pulso es una línea.

En la primera planilla obtuvimos:

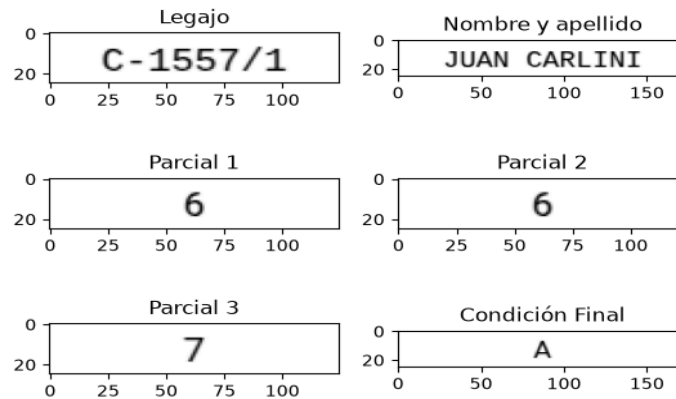
- 22 líneas horizontales (borde superior, fin del encabezado y 20 líneas que cierran cada registro).
- 8 líneas verticales (delimitan las 9 columnas).

Grosor de las líneas y heurística de recorte

Al observar los pares [inicio, fin] generados por las diferencias de las sumas, notamos un patrón estructural constante: en todas las líneas detectadas el inicio y el fin coinciden. Esto nos permitió deducir de forma heurística que todas las líneas divisorias del formulario miden exactamente 1 píxel de grosor. Lo comprobamos iterando sobre todas las imágenes de la carpeta.

Con las coordenadas de las líneas establecidas, cada celda queda delimitada espacialmente. Aprovechando nuestra heurística del grosor de 1 píxel, ajustamos los límites de recorte (desde el píxel siguiente a una línea horizontal hasta el anterior a la siguiente). Esta lógica matemática garantiza que la tinta de la cuadrícula quede estrictamente fuera del recorte de la celda, evitando generar falsos caracteres o ruido indeseado en el análisis de las componentes conectadas.

Primer fila de registros



Detección de caracteres con componentes conectadas

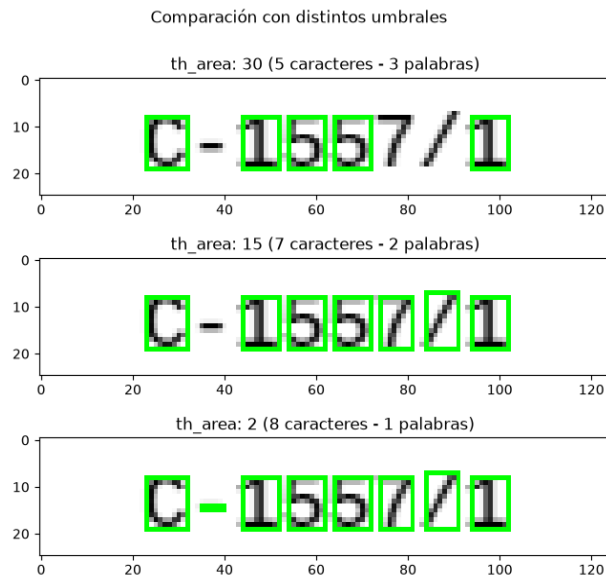
Para contar los caracteres de cada celda usamos `cv2.connectedComponentsWithStats`, que etiqueta cada grupo de píxeles conectados y devuelve una fila de stats con `[x, y, ancho, alto, área]`. Cada componente conectada corresponde a un carácter.

Encontramos varios obstáculos:

- La función considera objeto a todo lo distinto de 0, por lo que fue necesario binarizar con letras = 1, fondo = 0.
- La fila 0 de stats siempre corresponde al fondo por lo que se descartó.

Elección del umbral del área (`th_area`):

- Con `th_area = 30` se detectaban la mayoría de las letras, pero se perdían los símbolos.
- Con `th_area = 15` se incluía la "/" (área ≈ 19 px), pero no el guión.
- Con `th_area = 2` se detectaban los 8 caracteres. El guión tiene un área de apenas 5 píxeles (5x1), mientras que letras y dígitos están entre 19 y 86 píxeles. Un umbral tan bajo igual descarta píxeles sueltos que puedan generar ruido.



Por último ordenamos los caracteres por su coordenada x para recorrerlos de izquierda a derecha.

Conteo de palabras

Los espacios no representan componentes conectadas, por lo que no se pueden contar directamente. En su lugar, medimos el espacio entre cada caracter y el siguiente, la diferencia entre el inicio de un caracter y el fin del anterior ($x + \text{ancho}$).

Verificando en las planillas observamos que el espacio entre las letras de una misma palabra es de 0 a 4 píxeles mientras que entre palabras es de 10 a 13 píxeles.

Validación de campos

Con la cantidad de caracteres y palabras de cada celda, generamos una función que aplica las restricciones según el nombre del campo.

Clasificación de la Condición Final

Para el punto b) se requiere saber si es "L", "R", o "A". Optamos por un enfoque topológico basado en el conteo de agujeros (huecos cerrados) y el análisis de la columna izquierda de la letra.

Durante el desarrollo, nos topamos con un problema al intentar clasificar la letra "R". Al extraer las componentes conectadas de la celda invertida (donde la letra es el objeto y el fondo pasa a ser 0), si algún trazo de la letra tocaba el borde de la caja contenedora, el fondo exterior se fragmentaba en múltiples componentes independientes, alterando por completo el conteo matemático de agujeros y arrojando clasificaciones erróneas.

Para superar este obstáculo, nos apoyamos en el uso de Inteligencia Artificial (IA), la cual nos sugirió agregar un borde artificial de 1 píxel alrededor de la sub-imagen mediante la función `cv2.copyMakeBorder`. Este margen actúa como un puente que unifica todo el fondo exterior en una sola componente conectada inquebrantable. Restando 2 al total de etiquetas obtenidas (una por el fondo unificado y otra por el valor neutro), obtuvimos el número exacto de agujeros internos, logrando la siguiente matriz de decisión:

Letra	Agujeros	Columna izq completa
L	0	sí
R	1	sí
A	1	no

Finalmente unificamos todas las funciones en `procesar_planilla` que para cada registro de una planilla hace lo siguiente:

- Recorta cada campo, cuenta caracteres y palabras, lo valida e imprime el resultado por terminal con formato `> Legajo: OK`, etc.
- Guarda la fila de resultados para el CSV.
- Si todos los campos del registro son OK, clasifica la Condición Final. Si es L o R:
 - Recorta el campo Nombre y Apellido.
 - Le agrega a la derecha una etiqueta escrita con `cv2.putText`: “LIBRE” en rojo o “RECUPERA” en naranja.
 - Une ambas partes con `np.hstack`.

Uso de Inteligencia Artificial y Librerías Externas

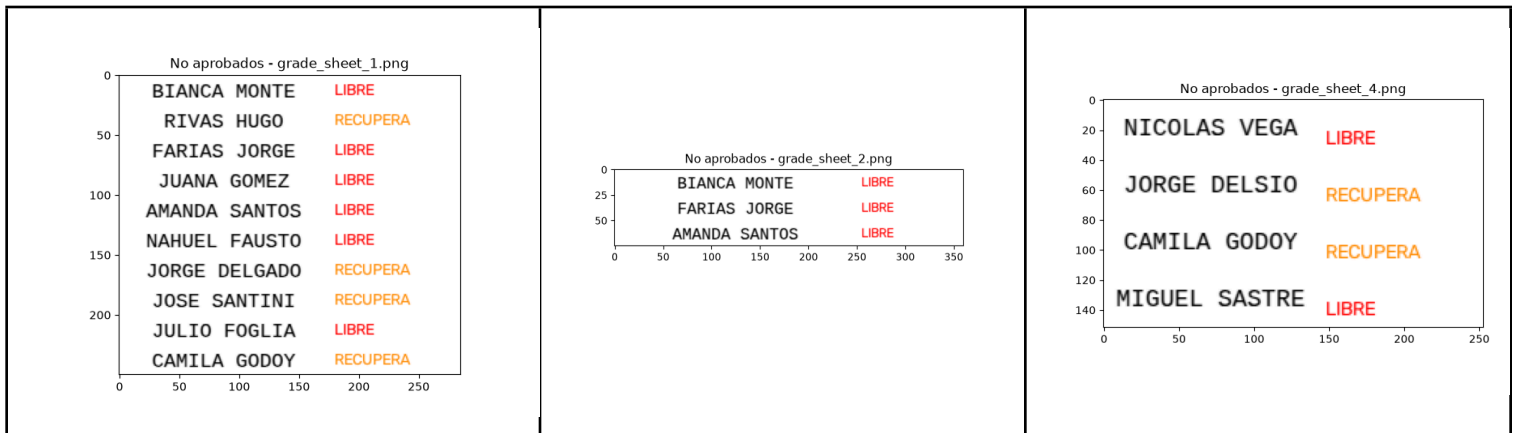
Si bien la lógica central de procesamiento se desarrolló analíticamente, declaramos la utilización de IA Generativa como soporte para resolver cuestiones de sintaxis y flujos de trabajo específicos:

- **Problema de la letra R:** La estrategia topológica mencionada anteriormente de aplicar un padding constante para unificar el fondo exterior.
- **Ensamblaje del mosaico de salida:** Asistencia en la sintaxis de las funciones `np.hstack` y `np.vstack` para concatenar matrices de imágenes, permitiendo unir los recortes de los apellidos con las etiquetas generadas en un único bloque visual que apila a todos los no aprobados de la planilla.

- **Iteración de archivos:** Uso de la librería externa **os** por practicidad para recorrer la carpeta de input e iterar sobre todas las planillas automáticamente.

Generación de Reportes

Al terminar la planilla, las filas de no aprobados se apilan en una única imagen de salida. A su vez, se guarda el CSV con una fila por registro. La primera columna es el ID y las seis columnas siguientes (en el orden de la planilla) contienen la validación "OK" o "MAL" de cada campo.



	A	B	C	D	E	F	G	H
1	ID	Legajo	Nombre y apellido	Parcial 1	Parcial 2	Parcial 3	Condición Final	
2	1	OK	OK	OK	OK	OK	OK	
3	2	OK	OK	OK	OK	OK	OK	
4	3	MAL	MAL	MAL	MAL	MAL	MAL	
5	4	OK	OK	OK	OK	OK	OK	
6	5	OK	OK	OK	OK	OK	OK	
7	6	OK	OK	OK	OK	OK	OK	
8	7	OK	OK	OK	OK	OK	OK	
9	8	OK	OK	OK	OK	OK	OK	
10	9	OK	OK	OK	OK	OK	OK	
11	10	OK	OK	OK	OK	OK	OK	
12	11	MAL	MAL	MAL	MAL	MAL	MAL	
13	12	OK	OK	OK	OK	OK	OK	
14	13	OK	OK	OK	OK	OK	OK	
15	14	MAL	MAL	MAL	MAL	MAL	MAL	
16	15	MAL	MAL	MAL	MAL	MAL	MAL	
17	16	OK	OK	OK	OK	OK	OK	
18	17	OK	OK	OK	OK	OK	OK	
19	18	OK	OK	OK	OK	OK	OK	
20	19	OK	OK	OK	OK	OK	OK	
21	20	MAL	MAL	MAL	MAL	MAL	MAL	
22								
23								

Conclusiones

El desarrollo del sistema de validación automatizada permitió procesar de manera integral las planillas de calificaciones, destacando los siguientes aspectos clave:

- **Automatización y eficiencia del flujo:** Se logró un script completamente automático para la segmentación, extracción de datos, validación de reglas de formato y generación de reportes consolidados (imágenes de no aprobados y archivo CSV).
- **Robustez en la detección de la estructura:** El uso de proyecciones por sumas de filas y columnas, sumado a la heurística del grosor de línea, garantizó una delimitación precisa de cada celda y evitó la inclusión de bordes o artefactos que perjudicarán la segmentación de texto.
- **Clasificación topológica de alta precisión:** La combinación de componentes conectadas, estimación de espacios entre caracteres y el análisis de agujeros internos permitió distinguir de forma exacta la Condición Final (L, R, A) y validar rigurosamente cada campo.